

Model interface

A hands-on course in building agents by building their harness

Lesson 1

The model is a function

TokensOut = Model(TokensIn)

- Tokens go in as a **request**; tokens come back as a **response**
- Just like any other API endpoint
- The **model interface** is how the harness makes that call

The model interface is the parentheses

Agent = Harness(Model)

- **Control flow** decides when to call
- **Input** gathers what goes in
- **Context** decides how it's presented in the request
- **Output** handles the response

The whole of quark.py: one call (abridged)

```
from anthropic import Anthropic

# — model interface —————
client = Anthropic()
MODEL = "claude-sonnet-5-5"
def call(each=lambda event: None, **request):
    with client.messages.stream(model=MODEL, **request) as stream:
        for event in stream: each(event)
    return stream.get_final_message()

output = call(max_tokens=16384, messages=[...])
print(output.model_dump_json(indent=2))
```

The `messages` list holds one `user` message: "What's in this directory?"

One function, one place

- `client = Anthropic()` : where the request goes, Anthropic's hosted API through the official SDK
- `call()` : the model interface, all of it. Every later lesson calls the model through it
- `messages=[...]` : the tokens in, each with a `role` and `content`
- `max_tokens` : caps the tokens out, even mid-sentence
- `output` : the response, printed whole

The response streams back

- `client.messages.stream(...)` : the response arrives piece by piece, as it's produced
- A long response that arrives all at once can time out while you wait; one that streams can't
- `for event in stream: each(event)` : each piece goes to `each()` as it arrives; here, `each` ignores them
- `stream.get_final_message()` : the whole response, put back together

Run it

```
uv run lessons/01-model-interface/quark.py
```

```
{  
  "content": [  
    {  
      "thinking": "",  
      "type": "thinking"  
    },  
    {  
      "type": "text"  
    }  
  ],  
  "stop_reason": "end_turn",  
}
```

One real run (shortened)

Three fields matter

- **content** : the tokens out, as a list of blocks: first its `thinking` , then the `text` of its reply
- Thinking comes back empty by default, with only a `signature` , an encrypted copy only the API can read
- **stop_reason** : `end_turn` means it finished; `max_tokens` means it hit the cap
- **usage** : `input_tokens` 15 is TokensIn, `output_tokens` 240 is TokensOut, thinking included

It knows the next step. It can't take it.

The `text` block, in part:

I can't see your directory. I don't have access to your file system in this conversation...

macOS/Linux: `ls` (or `ls -la` to include hidden files and details)

What else a model interface can be

- **Where the request goes, and how:** hosted API, your own machine, a gateway; SDK or raw HTTP
- **Which model answers:** one model, or a backup when the first is unavailable
- **How the response arrives:** all at once, or streamed so it can't time out
- **When a request fails:** retry, wait out a rate limit, give up after a timeout
- **Settings and volume:** tokens out, how much it thinks, one request or a batch

A gateway is this primitive alone

- [LiteLLM](#) and [OpenRouter](#) are the model interface and nothing else
- A harness sends them one request format
- They handle where it goes, which provider and model answer, backups, retries and rate limits
- A harness that uses one has handed off its model interface

model_interface.py does more (abridged)

```
client = anthropic.Anthropic(timeout=120, max_retries=3)
MODELS = ["claude-sonnet-5-5", "claude-opus-5-5"]

def call(messages, max_tokens=16384, effort="high", thinking="summarized"):
    for model in MODELS:
        try:
            with client.messages.stream(model=model, ...) as stream:
                return stream.get_final_message()
        except (anthropic.APIConnectionError, ...):
            continue
    raise RuntimeError("no model answered")
```

What it adds, mechanism by mechanism

- **When a request fails:** `timeout=120`, and `max_retries=3` retries with longer waits
- **Which model answers:** if every retry fails, the next model in `MODELS`
- **How the response arrives:** it streams, like `quark.py`; `get_final_message()` hands back the whole response
- **Settings:** `max_tokens`, `effort` and `thinking` are arguments; `"summarized"` asks for a summary of the thinking

Its run: the thinking now has text

```
uv run lessons/01-model-interface/model_interface.py
```

The `thinking` block, in part:

I don't actually have access to this user's file system, so I can't see what's in their directory...

The rest only shows when something goes wrong: a dropped connection, a rate limit, a model that's down.

The rule

The model interface sends tokens to the model as a **request** and gets tokens back as a **response**.

What it never does

- When to call is **control flow**
- Gathering what goes in, from a person or the world, is **input**
- How it's presented in the request is **context**
- What happens to the response is **output**

The model interface only gets the request there and the response back.

Both ends are stubs

```
"What's in this directory?" → request → model interface → response → print(...)
input (a stub)                                     output (a stub)
```

- The input is a hardcoded string no one typed
- The output dumps the whole response without handling any of it
- The model said what to do, and nothing could do it

Next: Input and output

A hands-on course in building agents by building their harness

Lesson 2